ARCHITECTURE SPECIFICATION

E-Commerce Frontend Architecture

Pure HTML, CSS, & JavaScript Ecosystem powered by LocalStorage

Document Type: System Architecture Design Document

Target Environment: Client-Side (Browser-Only Simulation)

Storage Engine: Web Storage API (LocalStorage)

Date: June 2026

1. Introduction & Overview

This document details the architecture and specifications for a fully functional client-side **E-Commerce Platform** built entirely on the **Frontend** using **HTML**, **CSS**, and **JavaScript**. By intentionally removing a traditional backend layer, the system leverages the browser's native **LocalStorage API** to handle persistent application states, simulating production workflows including user registration, authentication, live product catalogs, shopping cart dynamics, order creation, and profile histories.

The primary objective of this architecture is to emulate actual production-grade e-commerce applications within a decoupled, modular, browser-driven environment. It abstracts complex client-server paradigms into localized synchronous components, establishing a rigid baseline for structural front-end state management.

2. Architectural Framework (3-Layer Architecture)

To prevent spaghetti code and maintain high scalability, the architecture implements a strict separation of concerns through a localized 3-Layer paradigm:

1. UI Layer (User Interface)

Technologies: HTML & CSS.
Responsible for structural markup definitions and cross-device visual styles. It renders application data out to the viewport and forwards structural client events directly to the application logic layer.

2. Logic Layer (Application Controller)

Technologies: Vanilla JavaScript (ES6+ modules).
Acts as the programmatic controller orchestrating user actions. It handles events, updates application state, runs authentication barriers, calculates mathematical formulas, and coordinates layout repaints.

3. Data Layer (State Persistence)

Technologies: Browser LocalStorage Interface.
Fulfills persistent persistence roles by maintaining a clean JSON database representation directly in the user browser across sessions (Users, Products, Cart, Orders).

3. Project Structure & File Modularization

The physical layout of the repository separates concerns using granular script modules and structural pages. This modular scheme prevents logical contamination and simplifies long-term maintenance.

```
/ecommerce
│
├── index.html            // Home Page - Dynamic Product Feed & Category Filters
├── product.html          // Product Details Page - Granular Info & Cart Injectors
├── cart.html             // Cart Page - Line Items, Quantities, and Calculations
├── checkout.html         // Checkout Page - Billing, Shipping, and Final Validation
├── login.html            // Login View - Client Credentials Entry
├── register.html         // Register View - New Account Setup Matrix
├── profile.html          // Profile View - Customer Records & Historic Order Lists
│
├── /css
│   └── style.css         // Global Stylesheet & Responsive Layout Definitions
│
├── /js
│   ├── main.js           // Core Global Application Bootstrapper & UI Handlers
│   ├── auth.js           // Authentication Controller (Login, Register, Session
Guard)
│   ├── products.js       // Product Engine - Catalogs, Category Isolation, Search
│   ├── cart.js           // Shopping Cart State Controller & Total Estimators
│   ├── checkout.js       // Checkout Processor & Mock Order Factories
│   └── storage.js        // Low-level LocalStorage Getters/Setters Core Wrapper
│
└── /data (Optional)
    └── products.js       // Static Seed Array for Initial Product Populations
```

4. Core Platform Specifications

4.1 Authentication Architecture

- **Register:** Validates registration criteria, compiles credentials, and pushes new profiles into the global `users` dataset array.
- **Login:** Queries data tables to verify credentials. Upon authorization, records a transient session flag inside `currentUser` storage slots.
- **Logout:** Destroys the active `currentUser` storage key and forces immediate redirection to landing interfaces.

4.2 Home Page Execution Framework

- Dynamically reads the product catalog from storage blocks and parses it directly out onto responsive grids.
- Implements clean live text filtering across name arrays alongside rigid category classification sort metrics.

4.3 Product Specifications Object Schema

Field Name	Data Type	Description	Sample Value
`id`	String / Number	Unique identifier primary key	`101`
`name`	String	Commercial title display label	`"Premium Fragrance Oud"`
`price`	Float	Numeric base checkout value	`120.00`
`image`	String	Absolute file system pathway or CDN URI string	`"/assets/images/oud.jpg"`
`description`	String	Full structural marketing body statement text	`"High-end luxury perfume with oriental notes."`
`category`	String	Strategic classification token sorting filter	`"Fragrances"`

4.4 Shopping Cart Logic Matrix

- **Add Item:** Assesses existing product counts in `cart` arrays. Increments quantity attributes if match values pass true; else injects complete line payloads with baseline counters initialized to 1.
- **Remove Item:** Splits target indices directly out of active data arrays.
- **Quantity Modifiers:** Mutates item counts in real-time while updating total values across line boundaries.

4.5 Fake Checkout & Order Factory

- Captures customer records (Full Legal Name, Mailing Coordinates, Contact Lines).
- Processes mock billing paths (Cash payment or simulated Visa credentials validation matrices).
- Compiles complete state profiles into finalized `Order` arrays, flushes current cart slots, and records arrays to persistent logs.

4.6 User Profiles & Dashboard

- Renders dynamic profile readouts directly from current active `currentUser` pointers.
- Queries global historical order indices to isolate and display records tied to the authenticated ID.

5. Advanced Features & Technical Optimizations

To elevate implementation complexity and match production capabilities, the system architecture supports several optional mechanisms:

- **Wishlist Framework:** Parallels shopping cart routines to store bookmarked items.
- **Product Reviews Engine:** Stores multi-tier user feedback stars and description text tied directly back to individual product IDs.
- **Dark Mode State Management:** Persists user design style selections via native browser storage mechanisms.
- **Dynamic Pagination Core:** Fragments expansive feed nodes into predictable page slices.
- **Toast Notifications Wrapper:** Non-blocking micro UI notifications indicating successful system events.

6. Data Layer Architecture & Code Implementation

6.1 Storage Low-Level Helper Wrappers

Abstracting native system exceptions or raw serialization functions behind accessible utility methods simplifies application state logic.

```
function getData(key) {
    return JSON.parse(localStorage.getItem(key)) || [];
}

function setData(key, data) {
    localStorage.setItem(key, JSON.stringify(data));
}
```

6.2 Core Cart Mutation Sample

```
function addToCart(productId) {
    let cart = getData("cart");
    let item = cart.find(p => p.productId === productId);

    if (item) {
        item.quantity++;
    } else {
        cart.push({ productId, quantity: 1 });
    }

    setData("cart", cart);
}
```

6.3 Authentication Client Guard

```
let user = JSON.parse(localStorage.getItem(string"currentUser"));
if (!user) {
    window.location.href = "login.html";
}
```

7. User Journey Framework

The workflow details standard consumer paths from site initiation down to final client order validation sequences:

1. **Account Initialization:** Client establishes custom records through register forms.
2. **Session Sign-In:** Access credentials clear validation matrices and unlock restricted views.
3. **Dynamic Browsing:** User explores catalogs, filters categories, and executes live searches.
4. **Cart Injection:** Selected variants get pushed into active client state repositories.
5. **Order Validation:** Shipping, contact, and billing details are entered via the mock checkout.
6. **State Compilation:** Order payloads clear validation, flush active carts, and store deep history records.

CRITICAL SYSTEM DISCLAIMERS & DESIGN BOUNDARIES

- **Zero Real-World Security:** Because data operations bypass backends, all operations run inside client viewports. Code logic, records, and access tokens can be modified directly using web inspector suites.
- **Hard Storage Quotas:** LocalStorage bounds hard limits to approximately ~5MB across typical browsers. Large catalogs or heavy base64 imagery files will quickly cause resource allocation failures.
- **Simulated Checkout Realism:** Payment capture engines are mock visual representations and contain zero real monetary transmission pipelines.

8. Architectural Design Principles

- **Strict Component Separation (Modules):** Domain code blocks stay separated within individual scope files (e.g., `auth.js`, `cart.js`). Dom manipulations are isolated away from storage states.
- **Clean & Declarative Logic Design:** Avoid monolithic execution chains. Utilize pure functions and functional array paradigms (`.map()`, `.filter()`, `.reduce()`) to manage application state predictably.
- **Reusable Utility Blocks:** Write components once and use across pages to prevent logic duplication.

9. Implementation Strategy & Next Steps

For a reliable development lifecycle, implementation should follow a structured sequence to minimize state dependencies:

1. **Phase 1: Storage Wrapper & Authentication Layer (`storage.js`, `auth.js`)** - Build basic storage handlers and session controls first.
2. **Phase 2: Product Management Core (`products.js`)** - Initialize static seed inventories, map catalogs onto page layouts, and activate global product filter routines.
3. **Phase 3: Shopping Cart Engine (`cart.js`)** - Create line item state array monitors, wire dynamic layouts, and establish live price summation blocks.
4. **Phase 4: Order Pipeline & Profiles (`checkout.js`)** - Connect final mock checkout verification forms, record past purchases, and clean active checkout states.